In-Memory Object Storage in a Single Address Space

Taj Jethwani-Keyser
Harvard University

Abstract

We implement a prototype key-value database, SAS-DB, that loads clients as shared libraries into a single flat address space. Like prior in-memory object stores, our system achieves low latency by avoiding inter-process communication (IPC) for all reads and writes. However, unlike existing systems, SAS-DB does not suffer a performance penalty from multi-processing and it natively supports deep, pointer-rich data structures as objects. These properties, in addition to state-of-the-art synchronization primitives, allow SAS-DB to service dozens of clients at nanosecond latency. Our evaluations show that SAS-DB outperforms comparable systems by up to 5.68x on YCSB while providing a much more flexible API, though at the cost of strong process isolation.

1 Introduction

Modern data-intensive applications increasingly demand intra-server object storage at microsecond and even nanosecond latencies. Due to the prevalence of microservice architectures, which decompose monolithic applications into many independent programs, object stores play a crucial role in mediating access to shared state, even when all services are co-located. However, recent hardware studies argue that microsecond-scale stalls—of the kind incurred by every IPC—now dominate end-to-end response time in such systems [1, 7, 28]. Thus, intra-server communication requires different design considerations than in traditional remote databases.

For example, distributed machine learning frameworks such as Ray rely on a shared in-memory object store to pass tensors and Python objects between workers and across GPUs; Ray ships the Plasma store specifically to avoid copies on the intra-node hot path [17, 26]. In this setting, the bottleneck is no longer the disk or the network but the cost of exchanging live objects between processes on the same machine.

Most production in-memory databases were not designed to accommodate ultra-low latency. Both Redis [23] and Memcached [20], which have seen vast industry adoption, expose a socket-based interface: every `get` and `put` traverses the kernel, context-switches to a server thread, and copies data twice. This is acceptable in cloud scenarios where network latencies are in the tens of milliseconds, but the context-switching overhead is significant for local communication.

Turning to research prototypes, stores optimized for low-latency clusters such as RAMCloud [21] and FaRM [5] use kernel bypass and RDMA, respectively, and MICA [14] optimizes for high concurrency using data partitioning and zero-copy I/O; all three nonetheless presume a network and a serialized wire format. Plasma [26] stores all data in a shared memory region, achieving zero-copy queries across process boundaries; however, metadata operations still require IPC.

The closest prior system to ours, Lightning [28], recognizes that intra-server clients can avoid IPC entirely by mapping the *entire* store into client address spaces and operating on it through a specialized client library. However, Lightning serializes all operations through a single global spinlock and reports its best performance with a single client [28], sacrificing scalability for verifiable safety. One of our primary research objectives is to demonstrate when and how Lightning's performance can be improved, and to what extent Lightning's safety guarantees should be weakened.

Beyond synchronization, the multi-process model imposes an unavoidable cost: address translation. Shared memory regions on Linux are conventionally backed by 4 KiB pages and reachable through each client's own page table, so for working sets that exceed the few hundred translations cached in the Translation Lookaside Buffer (TLB), page table walks become a measurable fraction of overall memory access time [2, 18]. Pointer-rich data structures—graphs, trees, sparse matrices—aggravate the problem in two meaningful ways. First, each pointer chase potentially misses the TLB. Second, traditional shared memory cannot store raw pointers at all: every address embedded in a shared object must be a relative offset, forcing applications to rewrite their core data structures or marshal them on every `put`. Not only does this affect performance, but such an API is difficult to use correctly.

Single-address-space systems, in which all software shares

one virtual address space and protection is decoupled from translation, eliminate these problems by construction [3, 12]. Page tables are shared across all clients, huge pages are trivially usable, and pointers are universally meaningful. This allows us to realize the performance advantage of multi-threading in existing, multi-processing-reliant codebases, including object-store-backed microservices.

We revisit these ideas in user space. SAS-DB is a key-value store in which client code is loaded as a shared library into a single host process. Each `put` and `get` is an ordinary function call; values are referenced by raw pointers; and the hash table backing the store uses contemporary lock-free synchronization rather than a single spinlock. By giving up cross-client process isolation—which may be appropriate when client code is trusted—SAS-DB matches Lightning's avoidance of IPC while removing both its TLB pressure and its scalability ceiling. Our evaluation on YCSB shows a throughput improvement at every client count tested, with an API that admits arbitrary pointer-based objects that no comparable system supports. Additionally, our own microbenchmarks show a 12.4% throughput improvement at 64 concurrent clients over an equivalent shared memory-based implementation; the cost of isolation is nearly 10 million ops/s. As a result, we argue that single-address-space systems are worth adopting for applications seeking the highest possible performance.

2 Background

We provide a brief overview of in-memory object storage and the synchronization strategies used by concurrent hash tables, including fine-grained locking, lock-free algorithms, and safe memory reclamation (SMR). While these principles are well-studied in the literature, the single-address-space setting overturns many commonly-held beliefs about concurrent data structures. For instance, we have found that epoch-based reclamation (EBR) is *slower* than a well-optimized implementation based on hazard pointers, as the number of client threads is not known a priori. While our implementation of these primitives are not perfectly optimized, we hope that our experiences will be useful for the development of future single-address-space systems nonetheless.

2.1 In-Memory Object Stores

In-memory object stores allow independent programs to share live data without the overhead of a disk write. While multi-threaded applications enable data sharing by default (i.e., by memory reads and writes), *multi-processed* applications, where each component is executed in its own isolated process, require specialized systems to communicate. Oftentimes, such architectures are chosen to enable greater system security [24] and fault tolerance [27]. For example, the Google Chrome browser renders web sites in isolated processes to reduce the attack surface of malicious JavaScript code [24]. This use-case is incompatible with single-address-space systems; in general, it is typical to trade performance for security.

However, multi-process architectures may also be chosen for modularity (programs can be developed and compiled independently) and convenience (programs can be tested and run independently). In these settings, we wish to maintain the benefits of *logical* isolation, but depending on our security model and performance requirements, it may be desirable to allow for greater data sharing between programs. This motivates a design continuum, visualized in Figure 1, where object storage systems share increasing levels of state.

Client-server model. In the client-server model, clients and the database server are executed in isolated processes; they communicate over a network or Unix socket. While this offers the strongest security guarantees—namely that no process can corrupt the memory of any other process—as described in Section 1, IPC overhead is significant when executing all processes on the same machine.

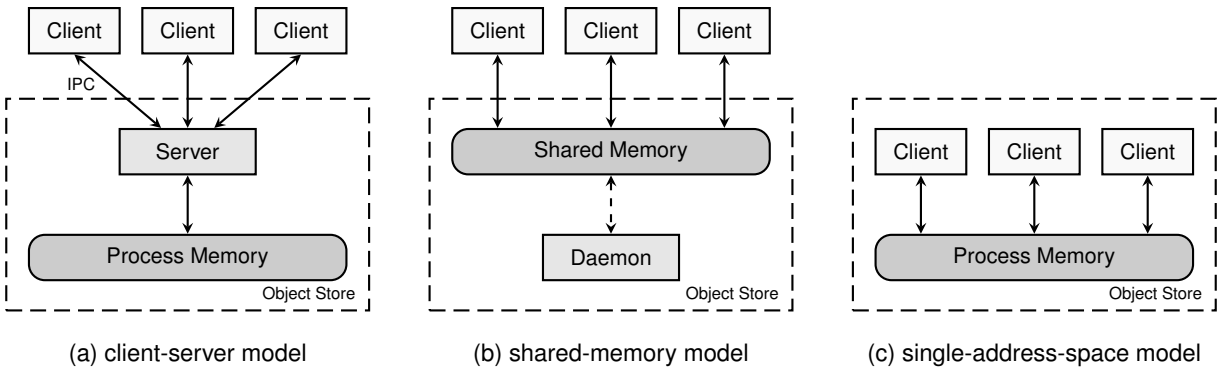
Shared-memory model. Pioneered by Lightning [28], the shared-memory model removes the designated server process and has clients perform transactions directly upon a shared memory buffer. Using write-ahead logging (WAL) and Intel Memory Protection Keys (MPK) [22], it is still possible to maintain isolation: MPK protects internal database state from faulty or malicious clients, and WAL ensures that transactions can be replayed in the case of a client crash. However, in order to simplify formal verification, Lightning uses a global spinlock to serialize transactions, and since MPKs can be overwritten in user space, additional methods are required to protect against actively malicious requirements. Thus, the shared-memory model may be considered an intermediate between fully-isolated clients and non-isolated clients.

Single-address-space model. In the single-address-space model, every program executes within the same address address space (in Linux language, each program is a *thread* within the same *process*). This simplifies pointer-sharing and alleviates TLB pressure—a known bottleneck in multi-processed systems [2, 18]—at the cost of process isolation. SAS-DB is the first single-address-space object store. Note that although isolation is weaker than the shared-memory model (e.g., file descriptors and signals are shared), Intel MPK can still be utilized to isolate clients from the database and from each other. Even within the single-address-space model, there are performance-isolation tradeoffs.

In this work, we operate within the single-address-space model due to its potential for maximal performance.

2.2 Concurrent Hash Tables

Since object stores typically expose a key-value interface and range queries across objects are not semantically meaningful, hash tables are a natural fit for scalable, low-latency queries. We borrow from a long line of work on optimizing hash tables



(a) client-server model (b) shared-memory model (c) single-address-space model

Figure 1: In-memory object store architectures. Figure inspired by Lightning [28].

for high-concurrency. While several open-source implementations are available online,[1] they are difficult to adapt to specific use-cases and often introduce large software dependencies (e.g., Facebook's Folly). As a result, our description focuses on techniques rather than specific implementations.

2.2.1 Lock Striping

Rather than using a single, global lock to synchronize access to the entire hash table, *lock striping* partitions the table by key into an array of smaller segments. Each segment is protected by a readers-writer lock [11]. In order to resize the table, one can acquire every lock, or, alternatively, acquire an exclusive lock over the entire table. The latter trades common-case latency for tail latency by introducing a shared lock to every operation. Lock striping scales well so long as the hot segment count exceeds the number of contending threads—in particular if the workload is ready-heavy—but the per-operation atomic on the readers-writer lock remains a fixed cost.

2.2.2 Lock-free Chaining

A concurrent data structure is *lock-free* if some thread is guaranteed to make progress in a bounded number of its own steps, so that no thread can stall the system as a whole by being descheduled or crashing [10]. While it is not true in general that lock-free algorithms outperform locking-based ones, they often offer superior tail latency and concurrency scaling. This makes them an obvious candidate for our use-cases.

In the absence of collisions, lock-free hash tables are very simple. One can implement insertion with the following compare-and-swap loop:

```
while (!CAS(&table[hash(key)], ptr, expected)) {
    yield();
}
```

[1] folly::ConcurrentHashMap, atomic_hash

A `get` simply performs an atomic read of the pointer `&table[hash(key)]`. Unfortunately, collisions are unavoidable, and resolving them in a lock-free table is substantially more involved. We discuss two influential designs. Michael's lock-free hash table [15] resolves collssion by chaining each bucket to a Harris-Michael lock-free linked list [8], with all of the insertion, lookup, and deletion operations expressible as CASes on node pointers. Shalev and Shavit's split-ordered list [25] goes further and avoids rehashing entries on resize: keys are stored in a single linked list whose order is invariant under bucket-array growth, so resize updates pointers into the list rather than moving entries. Our implementation uses the first design for simplicity; note that since all of our values are pointers, copy overhead is not quite as important.

2.2.3 Safe Memory Reclamation

Lock-free chaining introduces a subtle race condition during concurrent reads and writes. Suppose that thread t_1 reads a value pointer v just before thread t_2 replaces it with v'. If t_2 were to immediately free the memory pointed to by v, then t_1 would dereference a dangling pointer. A partial solution is to use reference counting, so that v will not be freed until every reader decrements the ref-count. However, it is still possible that t_2 overwrites v, decrements its ref-count to zero, and frees v before t_1 has a chance to increment the ref-count.

This problem also applies to the pointers internal to the data structure. A thread that has just read a node pointer may dereference that pointer arbitrarily later, so nodes cannot simply be freed once they are unliked. Safe memory reclamation (SMR) schemes give threads a principled way to defer reclamation of this memory until no concurrent reader can possibly hold a stale reference [9, 16].

Epoch-based reclamation (EBR). EBR [6] divides time into coarse epochs. A thread announces the current epoch when it enters a critical section and clears the announcement on exit; retired pointers accumulate in per-thread free lists and are freed only once every active thread has advanced past

the epoch in which the pointer was retired. The fast path is essentially free—a single relaxed store of the epoch number on entry—which makes EBR attractive in benchmarks dominated by short, regular critical sections. However, EBR makes strong assumptions regarding the liveness of threads within a critical section: if an active thread stalls, reclamation is deferred for *all* threads, and memory growth is unbounded.

Hazard pointers (HP). HP [16] tracks individual pointer references rather than whole epochs. This allows the system to bound memory consumption (i.e., $O(1)$ references per stalled thread) at the cost of a slower fast-path. Each thread publishes, in a small set of per-thread slots, the pointers it is currently dereferencing; a reclaimer scans these slots before freeing and defers reclamation of any pointer still hazardous to some thread. This requires a release-store and a re-read per protected dereference, leading to potentially poor performance when many pointers must be protected in sequence.

The choice between EBR and HP is typically informed by thread count and workload regularity. The single-address-space setting changes both: client threads are not introduced once at startup but at any time *by the clients themselves*. Since the database engine is not decoupled by client behavior, we must be careful to account for adversarial client behavior, e.g., sudden over-subscription due to thread spawns. This is pathological for EBR and motivates our adoption of HP.

3 Implementation

We now describe the SAS-DB architecture in detail, including our design decisions during implementation. In Section 4.1, we provide micro-benchmarks of system components to justify these choices. In Section 4.2, we provide a comparison of Lightning with our most well-rounded implementation.

Our system is a C++-23 prototype consisting of approximately 4500 lines of code. While our client library is written in C, all of our tests and benchmarks are written in C++ and we do not guarantee compatibility. We also provide a minimal Java API for compatibility with YCSB [4].

3.1 Client Loading

At startup, Lightning executes the *host* binary which is responsible for loading client programs. Currently, our system takes as input a list of files corresponding to client binaries (which must be `-fPIC` shared libraries); these are then loaded using `dlopen` and a distinguished `entry` function is called from a new thread. The host sleeps until every client terminates; unlike Lightning, there is no daemon.

In order to support dynamic process invocation, an alternative design is to read client binary names from a Unix socket. This has no performance implications except that the host must monitor for new clients to launch. We have not implemented this yet simply because it is not relevant to benchmarking.

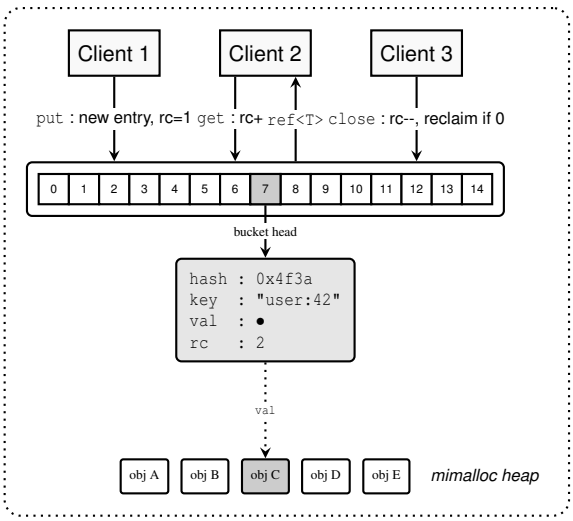


Figure 2: Backend of SAS-DB. Clients read and write directly to a concurrent hash table. Values initially have ref-count 1, which is incremented and decremented on `get` and `close`, respectively. Clients use a shared mimalloc heap.

3.2 Client API

Since clients are loaded as shared libraries, they have access to host code via symbol resolution. We provide the following API to clients (as a single-header include `client.h`) which dynamically executes host code at runtime:

- `get(key)` → `handle*`: Get from a key (which may be any C-string). Rather than returning values directly, this function returns an opaque handle which must be *dereferenced* using the following operator.
- `deref(handle*)` → `void*`: Dereference a handle to obtain a *value pointer*. All data types stored within SAS-DB are 8-byte pointers. This allows clients to define their own value representations, e.g., arrays of fields or pointer-based trees.
- `close(handle*)`: Decrement the ref-count of a handle. Necessary to prevent memory leaks.
- `put(key, value*, dtor)`: Store a value pointer at the location referenced by `key` and register a *destructor* to be called when the value's ref-count reaches zero. This function takes ownership of `value` via `dtor`.

3.2.1 Reference Counting

The decision to use reference counting for values was not made lightly. Since every value needs a corresponding handle, a handle must be allocated on every `put`. In addition, the atomic increment and decrement on every `get` and `close` can lead to frequent cache coherence misses under heavy contention. Unfortunately, every alternative either reduces API

flexibility or performance. For example, a visitation-based API would need to hold locks (or stay within a EBR or HP critical section) while executing client callbacks. This leads to a clunky programming model for database queries, which often depend on the result of previous queries.

In order to mitigate these performance concerns, we use mimalloc [13], a general-purpose allocator optimized for concurrent, ref-count-heavy workloads. In addition, we maintain a thread-local pool of handle allocations that are used when possible; freed handles return directly to the pool until it reaches capacity.

3.2.2 C++ Library

In addition to the more general C API, we provide a C++ convenience library that uses the RAII pattern and templates for simpler programming. For example, storing and retrieving an integer uses the following syntax, where `close` is automatic:

```
void entry() {
    auto value = std::make_unique<int>(2640);
    sas::put("example", value);
    auto handle = sas::get("example");
    assert(*value == *handle);
}
```

The client entry-point `entry` will always succeed without an assertion failure or memory leak; the `handle` object can be dereferenced similarly to a raw value; and the value destructor is passed within the type of `std::unique_ptr`.

3.3 Backend

The database itself is a concurrent hash table much like the strategies discussed in Section 2.2. We provide five modular backends which are benchmarked extensively in Section 4.1:

Spinlock. A naive implementation based on C++'s `std::unordered_map` with a spinlock for synchronization. This is our baseline (and performs similarly to Lightning).

Sharded. A highly-optimized implementation of lock striping based on `boost::concurrent_flat_map`. Unlike chaining-based solutions, the sharded backend uses open addressing and SIMD-accelerated probing. Uses a stop-the-world approach to resizing with exclusive locks.

Lock-free (EBR). A lock-free implementation using chaining and EBR for reclamation. Supports cooperative resizing.

Lock-free (HP). A lock-free implementation using chaining and HP for reclamation. Supports cooperative resizing.

Hybrid. A further optimized form of the sharded backend that performs writes (i.e., value pointer CASes) while holding a *read* lock rather than a write lock. This is only possible because SAS-DB stores exclusively pointer types (which, in turn, is a consequence of the shared-address-space model). However, HP must still be used to protect value pointers before incrementing the ref-count.

Despite the complexity of the lock-free solutions, we find that the hybrid backend performs best under the most varied situations. As a result, it is the default in our codebase.

4 Evaluation

We evaluate SAS-DB along two axes. In Section 4.1 we use microbenchmarks to (i) compare the five hash table backends introduced in Section 2.2, (ii) isolate the architectural contribution of the single-address-space model, and (iii) measure the overhead added by the SAS-DB client API on top of the raw backend. In Section 4.2 we compare SAS-DB end-to-end against Lightning on the YCSB benchmark.

All experiments run on a single CloudLab node with a two-way SMT Intel Xeon (24 physical cores at 2.795 GHz, 32 KiB L1, 512 KiB L2, 16 MiB shared L3 across each 6-core slice). Frequency scaling is disabled. Both SAS-DB and Lightning are compiled with `-O3` under C++23 and linked against mimalloc [13]. Unless otherwise noted, microbenchmarks use $N = 2^{22}$ keys, a 50% read / 50% write mix, Zipfian access with $\theta = 0.99$, and report steady-state aggregate throughput from Google Benchmark. Latency statistics come from per-thread histograms merged at the end of each run.

For YCSB, we use the Java workload generator from the upstream YCSB distribution [4]. SAS-DB hosts a single multi-threaded JVM as a client, while Lightning runs the default multi-process recipe in which each client process is its own JVM and the keyspace is partitioned across processes. Although a partitioned keyspace is highly favorable to a concurrent system, we were unable to run Lightning's benchmarking code on non-partitioned keys.

4.1 Microbenchmarks

4.1.1 Backend Comparison

Figure 3 reports read-write throughput for the five backends as the client thread count is swept from $T = 1$ to 64. The spinlock baseline collapses immediately: serializing every operation through a single critical section, throughput drops below half of its single-threaded value at two threads and below 1% of its single-threaded value by sixteen threads. This motivates our implementation of complex, multi-threaded data structures to eliminate this scaling bottleneck.

The two lock-free chained backends (EBR and HP) perform similarly across the sweep until $T = 64$, when EBR suddenly collapses. Although this may appear to be experimental noise, we have reproduced this effect with multiple reruns; it is well-known that EBR underperforms when threads are frequently preempted [19], such as when threads are oversubscribed. While HP performs very well overall, our implementation does not currently have support for deletion, which requires much heavier atomic operations while traversing each chain. The sharded backend is faster than

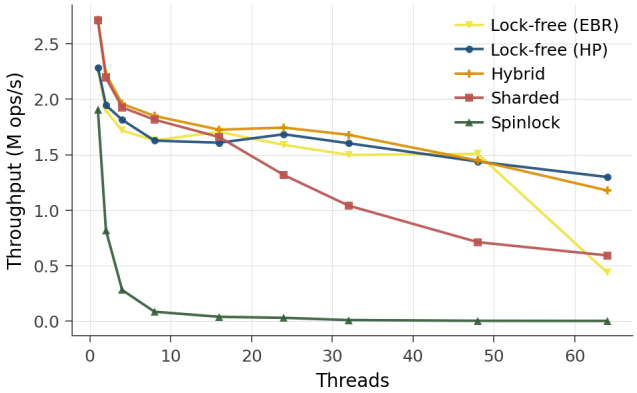


Figure 3: Mixed-workload throughput of the five SAS-DB backends as a function of client thread count ($N = 2^{22}$, 50/50 read/write, Zipfian $\theta = 0.99$).

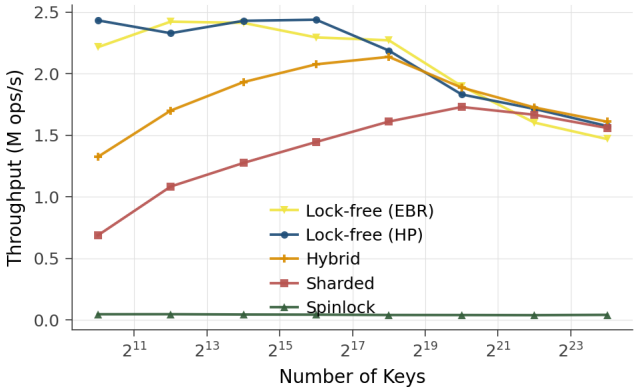


Figure 4: Mixed-workload throughput of the five SAS-DB backends as a function of the key count ($T = 16$, 50/50 read/write, Zipfian $\theta = 0.99$).

EBR/HP at low thread counts—SIMD-accelerated probing in `boost::concurrent_flat_map` dominates—but loses that advantage once contention increases.

Figure 4 reports read-write throughput as the number of keys is swept from $N = 2^{11}$ to 2^{24}. Although the lock-free backends perform significantly better at low key counts (possibly because they resize more aggressively than the sharded backend), at $N = 2^{24}$ every backend except for spinlock performs similarly at about 1.5 Mops/s.

The hybrid backend, which performs every pointer CAS while holding only a read lock, meets or exceeds every other backend at high thread counts and high key counts. We adopt it as SAS-DB's default.

4.1.2 Architecture Comparison

To isolate the architectural contribution of the single-address-space model, we run two builds that differ only in where the

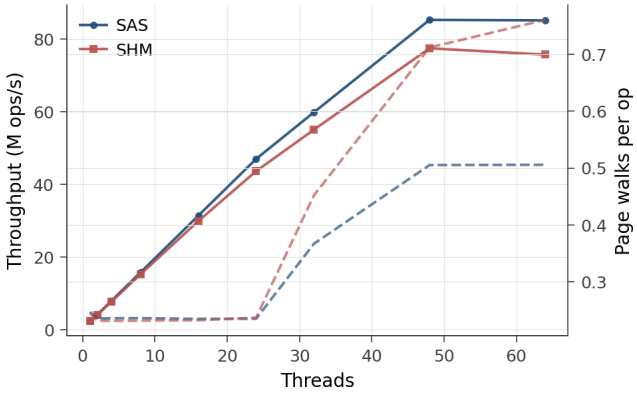


Figure 5: SAS-DB versus an otherwise-identical shared-memory build, as a function of client thread count. The right-hand axis reports per-op L2 TLB misses, sampled with `perf`.

store memory lives. The SAS build uses a standard shared heap; the SHM build maps a POSIX shared memory region into a leader process and a varying number of worker processes. In order to minimize reimplementation, we make use of mimalloc's *arena* feature, which allows users to provide pre-allocated heap regions (in this case, the shared memory buffer). While this suffices for benchmarking purposes, there is no support for cross-process `frees`, so we use a read-only workload. In addition, cross-process locking (i.e., for the hybrid backend) does not work, so both builds use the HP backend and the same workload generator.

Up to about 24 threads—the physical core count—the two configurations track each other within roughly 2%. Beyond that point, the workload saturates the available cores and processes begin to evict one another's TLB entries: the SHM build's per-operation page-walk count climbs sharpy (from 0.23 and 24 threads to 0.76 at 64 threads) as 64 distinct page tables walk the same data. The SAS build, sharing a single page table, sustains the same access pattern at 0.51 page walks per operation. The throughput gap follows the same trend: at 64 threads, SAS reaches 85.2 Mops/s vs. 75.7 Mops/s for SHM, a 12.4% improvement. That is close to 10 million additional ops/s attributable to collapsing the multi-process boundary alone.

4.1.3 Client API Overhead

The microbenchmarks above operate directly against the chosen backend. In order to sanity-check that dynamic loading and symbol resolution does not affect performance, we compare the native hybrid backend against the end-to-end SAS-DB stack on the same workload (i.e., the benchmark program is loaded as a shared library as opposed to linking `hybrid_store.cpp`). The end-to-end stack sustains throughput within a small factor of the native backend across the sweep and reports a read p99 of 1.20 μs and a write p99 of

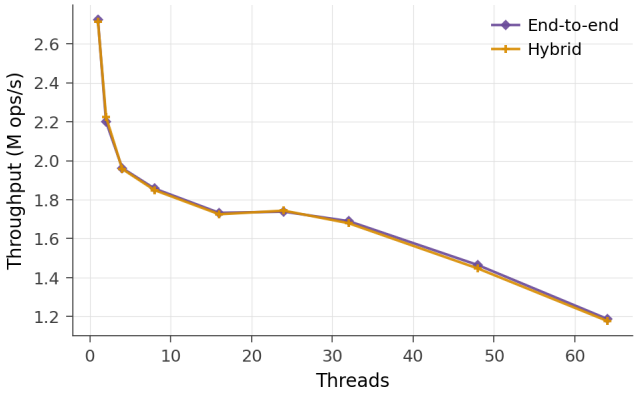


Figure 6: Throughput of the native hybrid backend versus the end-to-end SAS-DB stack. Both run the same workload.

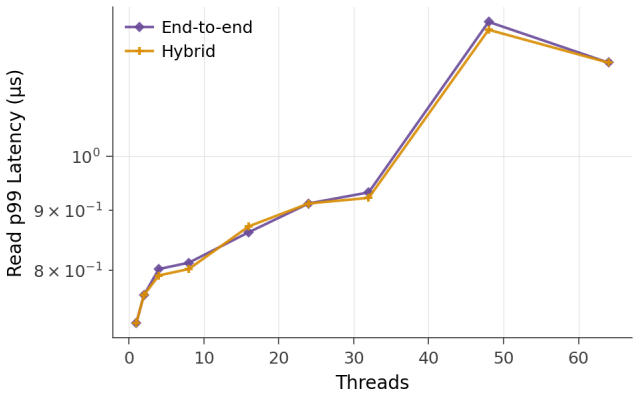


Figure 7: P99 read latency of the native hybrid backend versus the end-to-end SAS-DB stack. Both run the same workload.

1.37 μs. This is significantly more robust than comparable systems such as Lightning.

4.2 End-to-End Benchmarks

We compare SAS-DB to Lightning on the Yahoo! Cloud Serving Benchmark [4]. We run YCSB workloads A, B, C, D, and F at 1, 2, 4, and 8 client threads (although SAS-DB can support higher thread counts, the runtime of Lightning is prohibitively slow). Workload E (range scans) is omitted because neither system supports range queries. As described in the setup, Lightning runs in its recommended multi-process configuration with a partitioned keyspace, and SAS-DB runs as a single multi-threaded JVM client linked into the host.

SAS-DB outperforms Lightning on every workload at every thread count we tested. The smallest 8-thread speedup, 5.7×, occurs on the read-only workload C, which is the closest case to Lightning's best published configuration. The largest, 12.1×, occurs on the 50/50 mixed workload A, where Lightning's global spinlock serializes both reads and writes. Two characteristics of the data are worth highlighting. First, SAS-DB's run-phase throughput grows roughly linearly with thread count: across the workload set, 8-thread throughput averages 3.8× the single-threaded throughput, which is expected for a concurrent hash table with partitioned queries. Second, Lightning's run-phase throughput is essentially flat: its 8-thread numbers are within ±15% of its 1-thread numbers across all five workloads, consistent with the spinlock contention discussed in Section 1. The gap therefore widens monotonically with thread count, and SAS-DB's advantage is not the result of a particular workload but the underlying scaling behavior.

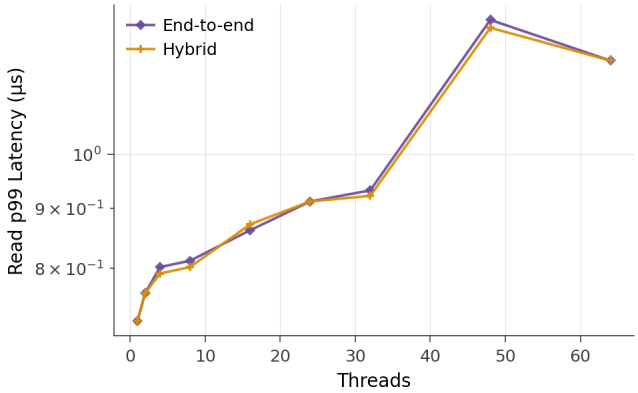


Figure 8: P99 write latency of the native hybrid backend versus the end-to-end SAS-DB stack. Both run the same workload.

	A	B	C	D	F
SAS-DB (8 threads)	1.42	1.43	1.52	1.29	1.04
Lightning (8 threads)	0.12	0.22	0.27	0.24	0.10
Speedup (×)	12.1	6.5	5.7	5.5	10.4

Table 1: YCSB run-phase throughput in Mops/s at 8 client threads. Workload E omitted (no range scans).

5 Conclusion

We presented SAS-DB, an in-memory key-value store that loads its clients as shared libraries into a single flat address space. Doing so eliminates both the IPC boundary that traditional client-server stores rely on for communication and the multi-process boundary that shared-memory stores rely on for isolation. On YCSB, SAS-DB outperforms Lightning by between 5.7x and 12.1x at eight client threads; on a microbenchmark that isolates the single-address-space contribution alone, the same hash table sustains 12.4% more throughput than an otherwise-identical shared-memory build at 64 threads. We also found that the right concurrency primitives in this settings are not the conventional ones: Epoch-based reclamation, commonly preferred for short-lived critical sections like hash table reads, is unsuited to a workload in which client threads spawn and stall outside of the engine's control. A simple

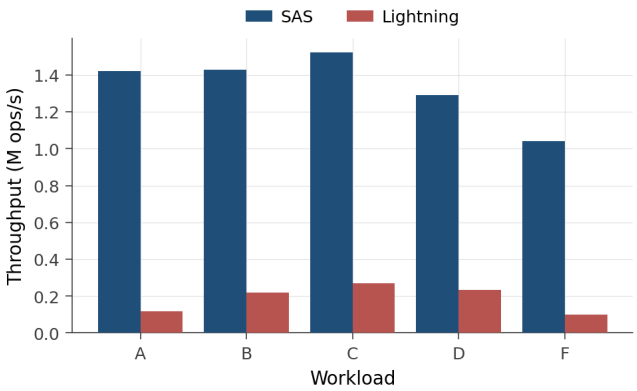


Figure 9: YCSB run-phase throughput at 8 client threads, SAS-DB versus Lightning.

hybrid scheme that performs pointer CAS while holding a read lock matches or beats every fully lock-free backend we implemented. We hope that the design choices and pitfalls documented here are useful for future single-address-space systems, and that the substantial performance overhead we measured motivates revisiting the long-standing assumption that intra-server object stores must isolate clients in separate processes.

References

[1] Luiz Barroso, Mike Marty, David Patterson, and Parthasarathy Ranganathan. Attack of the killer microseconds. *Communications of the ACM*, 60(4):48–54, 2017.

[2] Arkaprava Basu, Jayneel Gandhi, Jichuan Chang, Mark D. Hill, and Michael M. Swift. Efficient virtual memory for big memory servers. In *International Symposium on Computer Architecture (ISCA)*, 2013.

[3] Jeffrey S. Chase, Henry M. Levy, Michael J. Feeley, and Edward D. Lazowska. Sharing and protection in a single-address-space operating system. *ACM Transactions on Computer Systems*, 12(4):271–307, 1994.

[4] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with ycsb. In *Proceedings of the 1st ACM Symposium on Cloud Computing*, SoCC '10, page 143–154, New York, NY, USA, 2010. Association for Computing Machinery.

[5] Aleksandar Dragojević, Dushyanth Narayanan, Miguel Castro, and Orion Hodson. FaRM: Fast remote memory. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2014.

[6] Keir Fraser. *Practical Lock-Freedom*. PhD thesis, University of Cambridge Computer Laboratory, 2004.

[7] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, Kelvin Hu, Meghna Pancholi, Yuan He, Brett Clancy, Chris Colen, Fukang Wen, Catherine Leung, Siyuan Wang, Leon Zaruvinsky, Mateo Espinosa, Rick Lin, Zhongling Liu, Jake Padilla, and Christina Delimitrou. An open-source benchmark suite for microservices and their hardware-software implications for cloud and edge systems. In *Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.

[8] Timothy L. Harris. A pragmatic implementation of non-blocking linked-lists. In *International Symposium on Distributed Computing (DISC)*, pages 300–314, 2001.

[9] Thomas E. Hart, Paul E. McKenney, Angela Demke Brown, and Jonathan Walpole. Performance of memory reclamation for lockless synchronization. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2006.

[10] Maurice Herlihy. Wait-free synchronization. *ACM Transactions on Programming Languages and Systems*, 13(1):124–149, 1991.

[11] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2nd edition, 2020.

[12] Galen C. Hunt and James R. Larus. Singularity: rethinking the software stack. *SIGOPS Oper. Syst. Rev.*, 41(2):37–49, April 2007.

[13] Daan Leijen, Benjamin Zorn, and Leonardo de Moura. Mimalloc: Free list sharding in action. Technical Report MSR-TR-2019-18, Microsoft Research, June 2019.

[14] Hyeontaek Lim, Dongsu Han, David G. Andersen, and Michael Kaminsky. MICA: A holistic approach to fast in-memory key-value storage. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2014.

[15] Maged M. Michael. High performance dynamic lock-free hash tables and list-based sets. In *ACM Symposium on Parallel Algorithms and Architectures (SPAA)*, pages 73–82, 2002.

[16] Maged M. Michael. Hazard pointers: Safe memory reclamation for lock-free objects. *IEEE Transactions on Parallel and Distributed Systems*, 15(6):491–504, 2004.

[17] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 561–577, 2018.

[18] Juan Navarro, Sitaram Iyer, Peter Druschel, and Alan Cox. Practical, transparent operating system support for superpages. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2002.

[19] Ruslan Nikolaev and Binoy Ravindran. Snapshot-free, transparent, and robust memory reclamation for lock-free data structures. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI 2021, page 987–1002, New York, NY, USA, 2021. Association for Computing Machinery.

[20] Rajesh Nishtala, Hans Fugal, Steven Grimm, Marc Kwiatkowski, Herman Lee, Harry C. Li, Ryan McElroy, Mike Paleczny, Daniel Peek, Paul Saab, David Stafford, Tony Tung, and Venkateshwaran Venkataramani. Scaling Memcache at Facebook. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2013.

[21] John Ousterhout, Arjun Gopalan, Ashish Gupta, Ankita Kejriwal, Collin Lee, Behnam Montazeri, Diego Ongaro, Seo Jin Park, Henry Qin, Mendel Rosenblum, Stephen Rumble, Ryan Stutsman, and Stephen Yang. The RAMCloud storage system. *ACM Transactions on Computer Systems*, 33(3), 2015.

[22] Soyeon Park, Sangho Lee, Wen Xu, HyunGon Moon, and Taesoo Kim. libmpk: Software abstraction for intel memory protection keys (intel MPK). In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 241–254, Renton, WA, July 2019. USENIX Association.

[23] Redis Ltd. Redis: an open source, in-memory data store, 2024. https://redis.io.

[24] Charles Reis, Alexander Moshchuk, and Nasko Oskov. Site isolation: Process separation for web sites within the browser. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1661–1678, Santa Clara, CA, August 2019. USENIX Association.

[25] Ori Shalev and Nir Shavit. Split-ordered lists: Lock-free extensible hash tables. *Journal of the ACM*, 53(3):379–405, 2006.

[26] The Ray Project. The plasma in-memory object store. https://ray-project.github.io/2017/08/08/plasma-in-memory-object-store.html, Aug 2017. Accessed: 2026-05-09.

[27] Stephanie Wang, John Liagouris, Robert Nishihara, Philipp Moritz, Ujval Misra, Alexey Tumanov, and Ion Stoica. Lineage stash: fault tolerance off the critical path. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, SOSP '19, page 338–352, New York, NY, USA, 2019. Association for Computing Machinery.

[28] Danyang Zhuo, Kaiyuan Zhang, Zhuohan Li, Siyuan Zhuang, Stephanie Wang, Ang Chen, and Ion Stoica. Rearchitecting in-memory object stores for low latency. *Proceedings of the VLDB Endowment*, 15(3):555–568, 2022.